

Министерство образования и науки РФ
СанктПетербургский Политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

Отчёт по Курсовой работе №1

по дисциплине «Технологии разработки качественного программного обеспечения»

Выполнил:

студент гр. 5130904/10102

Богданов Н.Р.

Преподаватель:

Маслаков А. П.

Содержание

1	Введение	3
2	Общая информация о модульном тестировании	3
2.1	Цель и задачи тестирования	3
2.2	Используемые инструменты	3
2.3	Применяемые техники тестдизайна	3
3	Отчёт о покрытии кода	3
3.1	Арі модуль	4
3.2	Весь проект	4
4	Примеры применения техник тестдизайна	4
5	Процедура расширения тестового набора при добавлении нового блока кода	8
6	Выводы и рекомендации	8

1. Введение

В данном отчёте представлена информация по модульному тестированию разработанного программного продукта. Тестирование выполнялось для одного модуля: `api`.

2. Общая информация о модульном тестировании

2.1. Цель и задачи тестирования

- Достичь покрытия кода тестами не менее 80% для модуля.
- Обеспечить автоматический запуск тестов при сборке проекта с использованием соответствующего сборщика.
- Реализовать минимум 25 тестов.

2.2. Используемые инструменты

- **Фреймворки для тестирования:** `pytest`, `unittest` (`AsyncMock`), `FastAPI` `TestClient`.
- **Библиотеки для мокирования:** `unittest.mock`, `monkeypatch` (`pytest`), `MagicMock`, `patch`.
- **Технологии и методики тестдизайна:** классы эквивалентности, прогнозирование ошибок, граничные значения, таблица принятия решений, причинноследственный анализ, диаграммы состояний, попарное тестирование.
- **Система статической типизации и валидации:** `pydantic`.
- **Инструменты и библиотеки для работы с асинхронным кодом:** `pytest.mark.asyncio`, `async def`, `AsyncMock`.

2.3. Применяемые техники тестдизайна

1. Классы эквивалентности
2. Граничные значения
3. Причинноследственный анализ
4. Прогнозирование ошибок
5. Попарное тестирование
6. Диаграмма состояний
7. Таблица принятия решений

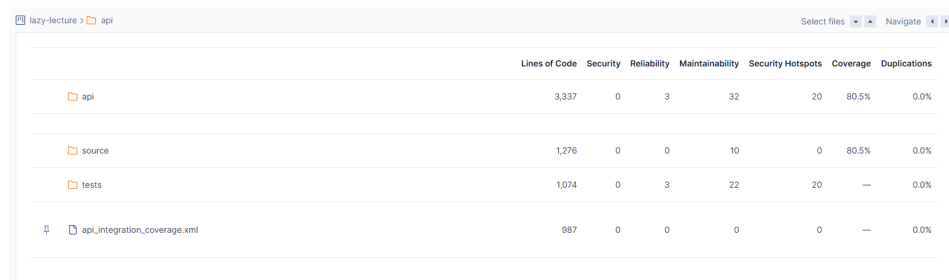
Примеры применения техник тестдизайна указаны в пункте 4.

3. Отчёт о покрытии кода

Для каждого модуля тестирования был получен отчёт о покрытии кода с использованием SonarQube.

3.1. Арі модуль

- Процент покрытия кода: 80.5%
- Скриншот отчёта:

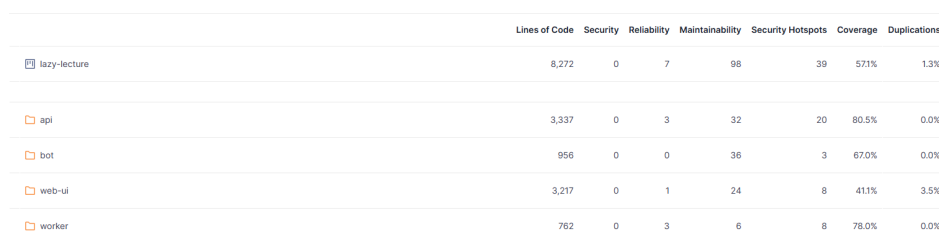


	Lines of Code	Security	Reliability	Maintainability	Security Hotspots	Coverage	Duplications
api	3,337	0	3	32	20	80.5%	0.0%
source	1,276	0	0	10	0	80.5%	0.0%
tests	1,074	0	3	22	20	—	0.0%
api_integration_coverage.xml	987	0	0	0	0	—	0.0%

Рис. 1: Отчёт SonarQube для покрытия арі

3.2. Весь проект

- Процент покрытия кода: 57.1%
- Скриншот отчёта:



	Lines of Code	Security	Reliability	Maintainability	Security Hotspots	Coverage	Duplications
lazy-lecture	8,272	0	7	98	39	57.1%	1.3%
api	3,337	0	3	32	20	80.5%	0.0%
bot	956	0	0	36	3	67.0%	0.0%
web-ui	3,217	0	1	24	8	41.1%	3.5%
worker	762	0	3	6	8	78.0%	0.0%

Рис. 2: Отчёт SonarQube для всего проекта

4. Примеры применения техник тестдизайна

Ниже приведены примеры примененных техник тестдизайна:

1. Классы эквивалентности

Разделение входных данных на группы (классы), которые ведут себя одинаково. До статочно протестировать одно значение из каждого класса, чтобы убедиться в кор ректной обработке всей группы.

```
1      Техника тестдизайна-: #1 Классы эквивалентности
2      Автор: Мирончук Юлиана
3      Описание:
4
5      * Для create_transcription есть четыре независимых входных
6      поля: creator_id, audio_len_secs, chunk_size_secs, current_state.
7      * Классы эквивалентности: валидный запрос не
8      ошибки порождает
9      @pytest.mark.asyncio
10     test_create_transcription_equivalence(): db = DummyDB()
11         req = TranscriptionRequest(
12             creator_id=10,
13             audio_len_secs=90.0,
```

```

11         chunk_size_secs=10.0,
12         current_state=TranscriptionState.QUEUED,
13     )
14     result = await services.create_transcription(req, db)
15     assert result is not None
16     assert result.creator_id == 10
17     assert result.audio_len_secs == 90.0
18     assert result.current_state == TranscriptionState.QUEUED
19 
```

Listing 1: Применение класса эквивалентности

2. Граничные условия

Тестирование значений на границах классов эквивалентности, так как именно там чаще всего случаются ошибки.

```

1     Техника тестдизайна-: #2 Граничные значения
2     Автор: Мирончук Юлиана
3     Описание:
4
5     * - Проверяем пароли на нижние и верхние границы длины и
6       состав символов.
7
8     @pytest.mark.parametrize(
9         "password",
10         [
11             "Ab1!",           # слишком короткий нижняя ( граница)
12             "abcd1234!",      # нет заглавных
13             "ABCD1234!",      # нет строчных
14             "Abcdefgh!",      # нет цифр
15             "Abcd1234",       # нет спецсимвола
16             "A",              # слишком длинный верхняя ( граница)
17         ], 257, *
18     )
19
20     def test_validate_pass_invalid(password):
21         with pytest.raises(ValueError) as exc_info:
22             validate_pass(password)
23         EXPECTED = (
24             "Password must be 8-256 characters long, include
25             Latin letters "
26             "(uppercase and lowercase), numbers, and at least
27             one special character."
28         )
29         assert EXPECTED.strip() ==
30         str(exc_info.value).strip()

```

Listing 2: Применение граничных условий

3. Причинноследственный анализ

Метод позволяет выявить влияние разных условий на поведение системы. Используется для тестирования сложной бизнеслогики.

```

1     Техника тестдизайна-: #3 Причинноследственный- анализ
2     Автор: Мирончук Юлиана
3     Описание:
4
5     * Проверяем, что generate_token создаёт два разных токена -
6       access и refresh -
7
8     * в зависимости от типа токена в
9       payload. влияет на содержимое токена.
10
11     *

```

```

8      * Если заданы корректные входные параметры, ожидаем корректный
      * результат.
9  @pytest.mark.asyncio
10 async def test_generate_token(monkeypatch):
11     def fake_jwt_encode(payload, key, algorithm):
12         token_type = payload["token_type"]
13         token_str = token_type.value.lower() if hasattr(token_type,
14 " value") else str(token_type).lower()
15         return f"encoded_{token_str}_{payload['user_id']}"
16     monkeypatch.setattr("source.app.auth.services.jwt.encode",
17 fake_jwt_encode)
18     token_dict = await generate_token(user_id=1, password_timestamp
19 =1234567890.0)
20     assert token_dict["access_token"] == "encoded_access_1"
21     assert token_dict["refresh_token"] == "encoded_refresh_1"

```

Listing 3: Применение причинноследственный анализ

4. Прогнозирование ошибок

Использование знаний о типичных ошибках и слабых местах системы для их целе направленного тестирования.

```

1      Техника тестдизайна-: #4 Прогнозирование ошибок
2      Автор: Мирончук Юлиана
3      Описание:
4
5      * Подменяем jwt.decode так, чтобы он выбрасывал JWTError,
6      * и проверяем корректную обработку ошибки – должна
7      вернуться пустая структура ({}).
8
9  @pytest.mark.asyncio
10 async def
11 test_decode_token_exception(monkeypatch): def
12 fake_decode(token, key, algorithms):
13     raise JWTError("error")
14 monkeypatch.setattr("source.app.auth.services.jwt.decode",
15 fake_decode)
16 result = await decode_token("dummy_token")
17 assert result == {}

```

Listing 4: Применение прогнозирования ошибок

5. Попарное тестирование

Оптимизация тестов при наличии множества параметров. Тестируются все возмож ные пары входных значений, но не все комбинации, что значительно сокращает чис ло тестов.

```

1      Техника тестдизайна-: #5 Попарное тестирование
2      Автор: Мирончук Юлиана
3      Описание:
4
5      * Проверка корректного создания токена с различными
6      комбинациями access_token, refresh_token и token_type.
7
8      * Вместо полного перебора используется метод
9      библиотеки itertools (itertools.combinations), который генерирует минимальный набор
10 тестов,
11
12 * покрывающий все возможные пары значений между
13 параметрами.
14
15 @pytest.mark.parametrize(
16     "access_token, refresh_token,
17     token_type", list(

```

```

10     AllPairs(
11         {
12             "access_token": ["access123",
13             "access456"], "refresh_token": ["refresh123",
14             "refresh456"], "token_type": ["custom",
15             "Bearer"],
16         }
17     ),
18 ),
19 )
20
21 def test_token_custom_token_type(access_token,
22 refresh_token, token_type):
23     token_data = {
24         "access_token": access_token,
25         "refresh_token": refresh_token,
26         "token_type": token_type
27     }
28     assert token.token_type == token_typepeNotFound))
29     token(**token_data)

```

Listing 5: Попарное тестирование

6. Диаграмма состояний

Техника диаграммы состояний используется, когда поведение системы можно пред ставить как конечный автомат — система может находиться в одном из нескольких состояний, и между ними происходят переходы в ответ на события или действия пользователя.

```

1     Техника тестдизайна-: #6 Диаграмма состояний
2     Автор: Мирончук Юлиана
3     Описание:
4
5     * Проверка обычного обновление состояния (QUEUED ->
6     IN_PROGRESS) без ошибок и без чанка.
7
8     *
9
10    @pytest.mark.asyncio
11    async def
12    test_update_transcription_state_ok(): db =
13    DummyDB()
14
15    update = TranscriptionStatusUpdateRequest(
16        transcription_id=1,
17        current_state=TranscriptionState.IN_PROGRESS,
18        new_chunk=None,
19    )
20
21    result = await services.update_transcription_state(update,
22    db) assert result.current_state ==
23    TranscriptionState.IN_PROGRESS

```

Listing 6: Диаграмма состояний

7. Таблицы принятия решений

Метод организации тесткейсов в виде таблицы, где условия и их комбинации определяют ожидаемый результат.

```

1     Техника тестдизайна-: #7 Таблицы принятия решений
2     Автор: Мирончук Юлиана
3     Описание:
4
5     * проверяет позитивный сценарий, где все
6     выполняется корректно, и результат — успешная аутентификация
7     и генерация новых токенов.
8
9     @pytest.mark.asyncio

```

```

6      async                                     def
7      test_authenticate_refresh_token_success(monkeypatch):
8      payload = {"user_id": 1, "token_type": TokenType.REFRESH, "
9      password_timestamp": 1234567890.0}
10
11      async def
12      fake_decode(token): return
13      payload
14
15      monkeypatch.setattr("source.app.auth.services.decode_token",
16      fake_decode)
17      user = dummy_active_user()
18      fake_db = AsyncMock()
19
20      async def fake_authenticate_token(user_id, password_timestamp,
21      db)
22      :
23      return user
24
25      monkeypatch.setattr("source.app.auth.services.authenticate_token",
26      fake_authenticate_token)
27
28      async def fake_generate_token(user_id,
29      password_timestamp): return {"access_token": "new_access",
30      "refresh_token": "
31      new_refresh"}
32
33      monkeypatch.setattr("source.app.auth.services.generate_token",
34      fake_generate_token)
35      result = await authenticate_refresh_token("dummy_token",
36      fake_db) assert result == {"access_token": "new_access",
37      "refresh_token": " new_refresh"}

```

Listing 7: Применение таблицы принятия решений

5. Процедура расширения тестового набора при добавлении нового блока кода

В случае если это совершенно новый алгоритм, необходимо создать новый тест, который будет покрывать его использование. Когда этот код часть уже тестируемой функции необходимо расширить тест, чтобы он покрывал новый функционал.

6. Выводы и рекомендации

- Достигнуто покрытие кода тестами на уровне 80.5% для модуля api и 57.1% для всего приложения.
- Написано более 25 тестов для api.
- Автоматизация сборки и тестирования позволяет оперативно выявлять ошибки и улучшать качество кода.